Assignment #4-1 · Web Application Development

도서관 관리 웹 애플리케이션 DB Schema 설계 보고서

학번 / 이름: 2022065250 문상철
과제명: Assignment #4-1 (DB Schema 설계)

1. 설계 의도 및 핵심 아키텍처 개요

본 설계는 도서관 관리 웹 애플리케이션(Assignment #4-2 구현 예정)의 모든 필수 요구사항과 추가 기능(비통계 2종 + 통계 2종)을 데이터 중복 및 이상 현상(Anomaly) 없이 지원하기 위해 **제3정규형(3NF) / BCNF**를 준수하여 설계되었습니다. 특히 명세서의 세부 제약 조건을 완벽히 충족하기 위해 다음 세 가지 핵심 아키텍처 원칙을 적용하였습니다.

핵심 설계 의사결정 (Design Highlights)
① **서적 메타데이터(Book)와 실물 도서 복본(BookCopy)의 1:N 분리:** 명세서에서는 *"도서관에 동일한 책이 여러 개 있을 수 있는데 차트 집계 시에는 종합해서 집계한다"*, *"동일한 책을 동시에 2권 이상 대출할 수 없다"*, 그리고 *"대출 기록에서는 동일한 책 중에서도 정확히 어떤 책을 대출했는지 확인할 수 있어야 한다"*고 규정하고 있습니다. 이를 해결하기 위해 책의 고유 서지 정보(제목, 저자, ISBN 등)를 담는 **Book** 엔티티와 개별 실물 자산(복본 고유번호, 청구기호, 상태)을 담는 **BookCopy** 엔티티를 1:N으로 분리하였습니다.
② **다중 카테고리(M:N) 정규화:** *"책은 동시에 여러 카테고리에 속할 수 있다"*는 조건을 만족하기 위해 **Book**과 **Category** 사이에 교차 릴레이션(Associative Table)인 **Book_Category**를 두어 다대다(M:N) 관계를 해소하였습니다.
③ **무결성 기반 대출·반납 및 연체(7일) 판별: Loan** 엔티티에 `loan_date`(대출일), `due_date`(반납기한 = 대출일+7일), `return_date`(실제 반납일, 미반납 시 NULL)를 기록하여, 중복 상태 컬럼 없이도 *현재 대출 중인 도서*(`return_date IS NULL`)와 *7일 이상 미반납 연체자*(`return_date IS NULL AND CURRENT_DATE > due_date`)를 항상 정확하게 판별합니다.

2. 각 Entity의 역할 및 필요성

도서관 관리 시스템의 데이터 모델은 총 **7개의 핵심 엔티티(Entity)**(다대다 관계 해소용 교차 테이블 포함 시 8개 릴레이션)로 구성됩니다.

1) Users (회원 및 관리자 계정)

필수 엔티티

- **역할:** 도서관 웹 서비스에 가입한 일반 사용자(USER)와 사서 관리자(ADMIN)의 계정 식별 정보, 암호화된 비밀번호, 이름, 권한을 저장합니다.
- **필요성:** ID/비밀번호 로그인 인증, 회원가입 처리(관리자 가입 시 10자리 학번 추가 코드 `2022065250` 검증 후 `role='ADMIN'` 부여), 관리자 전용 기능(서적/카테고리 관리, 전체 회원 목록 열람)의 접근 제어(RBAC), 그리고 사용자별 대출/예약/리뷰 이력을 식별하기 위해 필수적입니다.

`user_id (PK, VARCHAR) | password_hash (VARCHAR) | name (VARCHAR) | email (VARCHAR, UQ) | role ('USER'/'ADMIN') | admin_code (VARCHAR, Nullable) | created_at (DATETIME)`

2) Book (서적 메타데이터 / 서종)

필수 엔티티

- **역할:** 도서관이 소장한 '책의 종류(서종)' 단위의 고유 서지 정보(제목, 저자, 출판사, ISBN, 출간연도)를 저장합니다.
- **필요성:** 동일한 책이 여러 권 비치되더라도 제목·저자 등 서지 데이터가 중복 저장되지 않도록 정규화합니다. 또한 서적 검색/정렬(제목·저자 기준), 최근 3개월 인기 서적 차트의 도서별 종합 집계, 그리고 *"동일한 책을 동시에 2권 이상 대출 불가"* 규칙을 검증하는 기준 엔티티로서 필수적입니다.

`book_id (PK, INT) | isbn (VARCHAR, UQ) | title (VARCHAR) | author (VARCHAR) | publisher (VARCHAR) | published_year (INT) | created_at (DATETIME)`

3) BookCopy (실물 개별 도서 / 복본)

필수 엔티티

- **역할:** 특정 서적(Book)에 속하는 실제 물리적 개별 책 한 권 한 권의 고유 번호(`copy_id`), 청구기호(`call_number`), 대여 상태(`status`)를 관리합니다.
- **필요성:** 명세서의 *"대출 기록에서는 동일한 책 중에서도 정확히 어떤 책을 대출했는지 확인할 수 있어야 한다"*는 요구사항을 충족하기 위해 필수적입니다. 또한 서적 목록 페이지에서 *전체 보유 수량*(`COUNT(copy_id)`)과 *대출 가능 수량 및 대출 가능 여부*(`status='AVAILABLE'` 존재 여부)를 산출합니다.

`copy_id (PK, INT) | book_id (FK → Book.book_id) | call_number (VARCHAR, UQ) | status ('AVAILABLE'/'BORROWED'/'MAINTENANCE') | acquired_date (DATE)`

4) Category (도서 카테고리) & Book_Category (서적-카테고리 매핑)

필수 엔티티

- **역할:** 도서 분류 체계(컴퓨터공학, 문학, 경제 등)의 고유 정보를 관리하며, `Book_Category`를 통해 각 서적과 다대다(M:N)로 연결합니다.
- **필요성:** 관리자가 카테고리를 독립적으로 추가/제거할 때 서적 데이터가 유실되는 삭제 이상(Deletion Anomaly)을 방지하고, 한 권의 책이 동시에 여러 카테고리에 속할 수 있도록 하며, 카테고리별 검색·정렬 및 카테고리별 인기 차트 집계를 수행하기 위해 필요합니다.

`Category: category_id (PK, INT) | category_name (VARCHAR, UQ) | description (VARCHAR)` ‖ `Book_Category: book_id (PK, FK) | category_id (PK, FK)`

5) Loan (도서 대출 및 반납 기록)

필수 엔티티

- **역할:** 사용자(Users)가 특정 실물 복본(BookCopy)을 대출한 일시, 반납 기한(대출일+7일), 실제 반납 일시를 기록하는 트랜잭션 이력 엔티티입니다.
- **필요성:** ① 현재 대출 중인 도서 확인, ② 1인당 동시 최대 3권 대출 제한 검증, ③ 7일 이상 미반납 연체자 판별 및 대출 차단, ④ 사용자별 과거·현재 대출 기록 조회(실물 복본 식별 포함), ⑤ 최근 3개월 대출 기록 기반 인기 서적 차트 집계를 위해 핵심적으로 필요합니다.

`loan_id (PK, INT) | user_id (FK → Users.user_id) | copy_id (FK → BookCopy.copy_id) | loan_date (DATE) | due_date (DATE) | return_date (DATE, Nullable)`

6) Review (도서 리뷰 및 별점) & 7) Reservation (도서 대출 예약)

추가 기능 엔티티

- **Review 역할 및 필요성:** 회원이 읽은 서적(Book)에 대해 1~5점 별점(`rating`)과 한줄 서평(`comment`)을 남기는 엔티티로, 도서 품질에 대한 이용자 피드백 공유 및 추가 통계 기능인 *서적·카테고리별 평점 통계*를 산출하기 위해 필요합니다.
- **Reservation 역할 및 필요성:** 특정 서적의 모든 복본(BookCopy)이 대출 중일 때 회원이 해당 서적(Book)에 대기열 예약을 걸어두는 엔티티로, 반납 시 선착순(`reserved_at`)으로 대출 우선권을 부여하여 도서관 이용 편의성을 높입니다.

`Review: review_id (PK) | user_id (FK) | book_id (FK) | rating (1~5) | comment | created_at` ‖ `Reservation: reservation_id (PK) | user_id (FK) | book_id (FK) | status | reserved_at`

3. Entity 간 Relation 분석 및 설계 의도

각 엔티티 사이의 관계(Relationship), 대응수(Cardinality), 참여 제약조건(Participation) 및 설계 목적은 아래 표와 같습니다.

Relation 명칭	연결 Entity	카디널리티	관계의 의미 및 필요한 이유 (설계 의도)
Has_Copy (복본 보유)	Book — BookCopy	**1 : N** (부분 : 전체)	하나의 서적 종류(Book)는 여러 권의 실물 복본(BookCopy)을 가질 수 있으며, 모든 BookCopy는 반드시 하나의 Book에 속합니다. 동일 도서의 다중 소장 관리, 보유 수량 파악, 차트 종합 집계를 동시에 달성하기 위해 필요합니다.
Belongs_To (카테고리 소속)	Book — Category (교차: Book_Category)	**M : N** (부분 : 부분)	한 권의 책은 동시에 여러 카테고리에 속할 수 있고, 하나의 카테고리에는 여러 권의 책이 포함될 수 있습니다. 관계형 모델에서는 교차 테이블 `Book_Category(book_id, category_id)`를 통해 외래키 쌍을 복합 기본키로 구성하여 정규화합니다.
Borrows / Borrowed_In	Users — Loan — BookCopy	**1 : N** 및 **1 : N**	한 명의 사용자(Users)는 여러 건의 대출 기록(Loan)을 생성할 수 있고(1:N), 하나의 실물 복본(BookCopy)도 시간차를 두고 여러 번 대출될 수 있습니다(1:N). Loan이 Book이 아닌 BookCopy와 관계를 맺음으로써 *동일한 책 중 정확히 어느 실물 복본을 빌렸는지* 완벽히 추적합니다.
Writes / Evaluates	Users — Review — Book	**1 : N** 및 **1 : N**	사용자는 여러 서적에 리뷰를 작성할 수 있고, 하나의 서적(Book)은 여러 사용자로부터 리뷰를 받을 수 있습니다. 리뷰는 실물 복본 상태가 아니라 책의 내용 자체에 대한 평가이므로 Book 엔티티와 직접 관계를 맺으며, `(user_id, book_id)`에 UNIQUE 제약을 두어 중복 평점 조작을 방지합니다.
Requests / Targets	Users — Reservation — Book	**1 : N** 및 **1 : N**	사용자는 모든 복본이 대출 중인 서적에 대해 예약을 걸 수 있습니다. 특정 복본이 아닌 '해당 서적 중 가장 먼저 반납되는 임의의 복본'을 배정받는 것이 도서관 업무 규칙에 부합하므로 Reservation은 Book과 1:N 관계를 형성합니다.

4. 추가 기능 설계 (총 4가지: 비통계 기능 2가지 + 통계 기능 정확히 2가지)

과제 명세의 추가 기능 조건(*"명세 이외의 다른 기능 3가지 이상 추가, 그중 통계와 관련된 내용이 반드시 2가지(2가지보다 적거나 많을 수 없음) 포함"*)을 엄격히 준수하여, 실제 도서관 웹 서비스에 필수적이고 유용한 **비통계 기능 2가지**와 **통계 기능 정확히 2가지**를 설계하였습니다.

비통계 1 도서 리뷰 및 별점 시스템 (Review)

기능 설명: 해당 도서를 대출하여 읽은 이력이 있는 사용자가 1~5점의 별점과 서평을 작성·조회하는 기능입니다.
필요성 및 기대효과: 단순 서지 정보만으로는 알기 어려운 책의 난이도와 유익함을 다른 이용자와 공유할 수 있으며, Loan 이력 검증을 통해 실제 독자만 리뷰를 남기도록 신뢰도를 보장합니다.

비통계 2 대출 중 도서 예약 대기열 (Reservation)

기능 설명: 원하는 서적(Book)의 모든 복본(BookCopy)이 대출 중(BORROWED)일 때, 사용자가 예약 대기열에 등록하고 반납 시 우선 대출권을 부여받는 기능입니다.
필요성 및 기대효과: 인기 도서를 빌리기 위해 사용자가 무한정 새로고침하며 대기해야 하는 불편을 해소하는 실제 도서관의 핵심 필수 기능입니다.

통계 기능 1 (필수 2개 중 1) 서적 및 카테고리별 평점·리뷰 통계 분석

기능 및 설계 의도: 기본 명세의 '인기 서적 차트'는 최근 3개월간의 *대출 횟수(양적 지표)*만을 반영합니다. 이를 보완하기 위해 Review, Book, Book_Category, Category 릴레이션을 조인·집계하여 ① **각 서적별 평균 별점(AVG(rating)) 및 리뷰 수(COUNT(review_id))**와 ② **카테고리별 평균 만족도 평점 및 최고 평점 도서 현황**을 제공하는 질적 통계를 설계하였습니다. 별도의 중복 통계 테이블 없이 정규화된 Review 테이블에서 실시간 집계하므로 데이터 정합성이 100% 유지됩니다.

```
SELECT c.category_name, COUNT(DISTINCT b.book_id) AS book_cnt, COUNT(r.review_id) AS review_cnt,
ROUND(AVG(r.rating), 2) AS avg_rating
FROM Category c JOIN Book_Category bc ON c.category_id = bc.category_id JOIN Book b ON bc.book_id = b.book_id LEFT
JOIN Review r ON b.book_id = r.book_id GROUP BY c.category_id, c.category_name ORDER BY avg_rating DESC;
```

통계 기능 2 (필수 2개 중 2) 월별 대출·연체율 추이 및 이용자 독서 통계 (다독/기한준수 분석)

기능 및 설계 의도: Loan과 Users 릴레이션의 날짜 데이터(loan_date, due_date, return_date)를 기반으로 ① **월별 전체 대출 건수 및 연체 발생 비율(return_date > due_date 또는 미반납 기한 경과 비율) 추이 통계**와 ② **회원별 누적 독서량(다독 순위) 및 기한 내 반납 준수율 통계**를 제공합니다. 관리자에게는 도서관 운영 지표를, 사용자에게는 자신의 독서 습관 통계를 제공하는 핵심 통계 기능입니다.

```
SELECT DATE_FORMAT(loan_date, '%Y-%m') AS loan_month, COUNT(*) AS total_loans, SUM(CASE WHEN COALESCE(return_date,
CURRENT_DATE) > due_date THEN 1 ELSE 0 END) AS overdue_cnt,
       ROUND(SUM(CASE WHEN COALESCE(return_date, CURRENT_DATE) > due_date THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 1)
AS overdue_rate_pct FROM Loan GROUP BY DATE_FORMAT(loan_date, '%Y-%m') ORDER BY loan_month DESC;
```

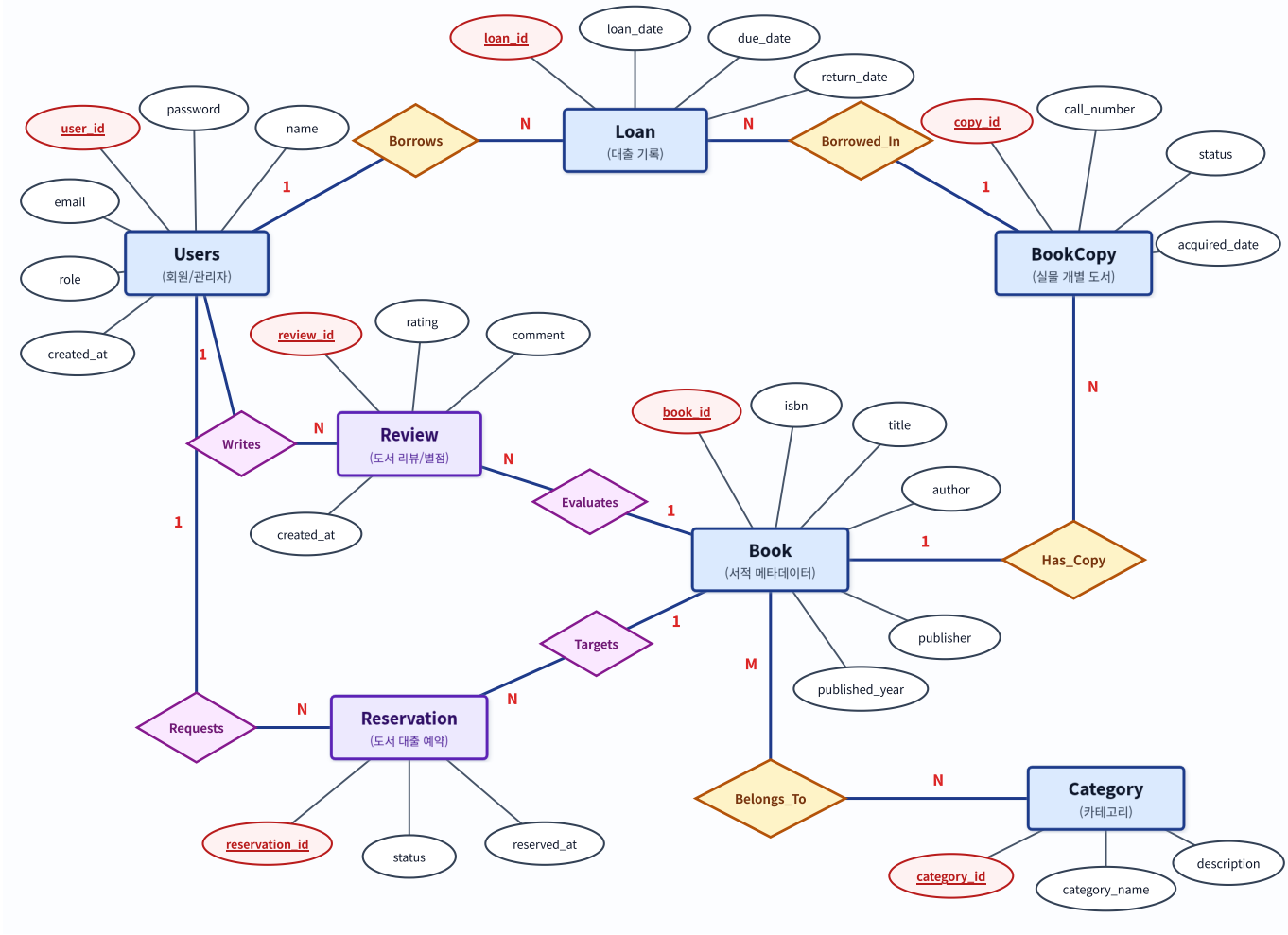
※ 필수 비즈니스 제약조건의 SQL/스키마 검증 매핑 요약

명세서 필수 규칙	설계된 스키마에서의 검증 및 처리 방식
동시 최대 3권 대출 제한	`SELECT COUNT(*) FROM Loan WHERE user_id = ? AND return_date IS NULL` 결과가 >= 3이면 대출 거부
동일 서적 동시 2권 이상 대출 불가	Loan과 BookCopy를 조인하여 해당 user_id의 미반납(return_date IS NULL) 대출 건 중 동일한 book_id가 존재하는지 확인 후 차단
7일 이상 미반납(연체) 시 대출 불가	`SELECT EXISTS(SELECT 1 FROM Loan WHERE user_id = ? AND return_date IS NULL AND CURRENT_DATE > due_date)`가 참이면 연체 상태로 판정하여 신규 대출 차단 (`due_date = DATE_ADD(loan_date, INTERVAL 7 DAY)`)
최근 3개월 인기 서적 차트 (전체/카테고리별, 복본 합산)	`Loan(loan_date >= CURRENT_DATE - INTERVAL 3 MONTH)` → BookCopy → Book으로 조인 후 Book.book_id 기준으로 GROUP BY하여 복본들의 대출 횟수를 서적 단위로 통합 집계

5. ER Diagram (Entity-Relationship Diagram)

표준 **Chen 표기법(Silberschatz 데이터베이스 교재 표준)**에 따라 작도한 전체 시스템의 ER Diagram입니다. 직사각형은 **Entity 집합**, 마름모는 **Relationship 집합**, 타원은 **Attribute**(실선 밑줄은 기본키 PK)를 나타내며, 연결선 위의 숫자와 기호(1, N, M)는 **Cardinality(대응수)**를 의미합니다.

[범례] ▭ **직사각형:** Entity 집합 ◇ **마름모:** Relationship 집합 ◯ **타원 (밑줄):** Attribute (Primary Key) **1, N, M:** 관계 대응수 (Cardinality)



[그림 1] 도서관 관리 웹 애플리케이션 전체 ER Diagram (Chen 표기법)

ER Diagram 핵심 구조 요약:
• **필수 영역 (청색 엔티티):** `Users`, `Loan`, `BookCopy`, `Book`, `Category`로 구성되며, `Book`(1) — `Has_Copy` — `BookCopy`(N) 분리를 통해 개별 복본 대출 이력과 도서 종류별 인기 차트 집계를 동시에 지원합니다.
• **추가 기능 영역 (보라색 엔티티):** `Review`(도서 평점/리뷰)와 `Reservation`(도서 예약 대기열)이 `Users` 및 `Book`과 각각 1:N 관계를 형성하며, 이 데이터와 `Loan` 기록을 기반으로 2가지 필수 통계(평점 통계, 월별 대출/연체율 통계)가 산출됩니다.

6. Relation Schema Diagram (관계 스키마 다이어그램)

ER Diagram을 관계형 데이터베이스 모델(Relational Model)로 변환한 **Relation Schema 그림**입니다. 다대다(M:N) 관계인 Belongs_To는 교차 릴레이션 **Book_Category**로 변환되었으며, 각 릴레이션의 **기본키(PK)는 실선 밑줄**로, **외래키(FK)는 참조하는 기본키(PK)를 향하는 실선 화살표(→)**로 명확히 표현하였습니다.



[그림 2] 도서관 관리 웹 애플리케이션 전체 Relation Schema Diagram (PK 밑줄 및 FK 참조 화살표 표기)

6.1 각 Relation 스키마 및 외래키(FK) 참조 무결성 제약조건 명세

Relation 이름	Relation Schema 표기 (PK, *FK*)	참조 무결성 (Foreign Key) 및 주요 제약조건
Users	Users(user_id, password_hash, name, email, role, admin_code, created_at)	• email UNIQUE, role IN ('USER', 'ADMIN')
Book	Book(book_id, isbn, title, author, publisher, published_year, created_at)	• isbn UNIQUE • 제목/저자 검색·정렬 인덱스
BookCopy	BookCopy(copy_id, *book_id*, call_number, status, acquired_date)	• *book_id* → Book(book_id) ON DELETE CASCADE • call_number UNIQUE
Category	Category(category_id, category_name, description)	• category_name UNIQUE NOT NULL
Book_Category	Book_Category(*book_id*, *category_id*)	• *book_id* → Book(book_id) ON DELETE CASCADE • *category_id* → Category(category_id) ON DELETE CASCADE
Loan	Loan(loan_id, *user_id*, *copy_id*, loan_date, due_date, return_date)	• *user_id* → Users(user_id) ON DELETE RESTRICT • *copy_id* → BookCopy(copy_id) ON DELETE RESTRICT
Review	Review(review_id, *user_id*, *book_id*, rating, comment, created_at)	• *user_id* → Users(user_id), *book_id* → Book(book_id) • UNIQUE(user_id, book_id), CHECK (rating BETWEEN 1 AND 5)
Reservation	Reservation(reservation_id, *user_id*, *book_id*, status, reserved_at)	• *user_id* → Users(user_id), *book_id* → Book(book_id) • status IN ('WAITING', 'FULFILLED', 'CANCELLED')